

Федеральное государственное автономное образовательное учреждение
высшего образования

Национальный исследовательский университет

ИТМО

Факультет информационных технологий и программирования

ОТЧЕТ

по лабораторной работе №2

на тему:

«Модификации градиентного спуска»

Работу выполнили студенты

Группы М3233

Калиновский Арсений Владимирович

Рудаков Максим Андреевич



ITMO UNIVERSITY

Санкт-Петербург

25 июня 2026 г.

Цели работы

Целью лабораторной работы является изучение методов решения задач регрессии. В ходе работы исследуются разные подходы к минимизации функции потерь при обучении линейных и полиномиальных регрессионных моделей, а также анализируется влияние регуляризации и параметров оптимизации на качество получаемого решения.

Задачи, решаемые при выполнении работы

- реализовать линейную и полиномиальную регрессию
- реализовать аналитический метод решения задачи линейной регрессии
- реализовать итерационные методы оптимизации: стохастический градиентный спуск (SGD), mini-batch градиентный спуск, метод Гаусса–Ньютона и метод Левенберга–Марквардта
- реализовать механизмы регуляризации L_1 , L_2 и Elastic Net
- провести сравнение методов оптимизации по скорости сходимости и качеству получаемого решения
- исследовать влияние размера мини-пакета на процесс обучения
- исследовать влияние различных видов регуляризации на качество аппроксимации и устойчивость модели

Объект исследования

Объектом исследования являются методы численной оптимизации, которые используются для решения задач регрессии.

В качестве тестовых данных рассматриваются сгенерированные самостоятельно выборки, полученные на основе заранее заданных функций с добавлением случайного шума. Для аппроксимации данных используются линейные и полиномиальные регрессионные модели разной степени.

Также рассматривается влияние метода оптимизации, параметров обучения и способов регуляризации на процесс минимизации функции потерь и качество итоговой модели.

Тестовые функции

В соответствии с рекомендациями методических указаний были выбраны две аналитические зависимости различной сложности.

Для моделирования почти линейной зависимости использовалась функция

$$f_1(x) = 2x + 1 + 0.1 \sin(5x).$$

Основную часть зависимости задаёт линейный тренд, а синусоидальная составляющая создаёт небольшое нелинейное отклонение.

Для моделирования существенно нелинейной зависимости использовалась функция

$$f_2(x) = \sin(2x) + 0.2x^3 + \exp(-(x-1)^2).$$

Функция содержит периодическую компоненту, полиномиальный тренд и локальный пик, что соответствует рекомендациям лабораторной работы и позволяет исследовать возможности полиномиальной регрессии различных степеней.

Для обеих функций данные генерировались на интервале

$$x \in [-3, 3],$$

после чего к значениям функции добавлялся нормально распределённый шум

$$\xi \sim \mathcal{N}(0, 0.2^2).$$

Итоговые наблюдения формировались по формуле

$$y_i = f_{\text{true}}(x_i) + \xi_i.$$

Модели задач

- линейная регрессия (полиномиальная степени 1)

$$f(x, w) = w_0 + w_1x$$

- полиномиальная регрессия степени d

$$f(x, w) = \sum_{k=0}^d w_k x^k, \quad \text{где } d \in \{2, 3, 4, 5\}$$

В качестве функции качества используется среднеквадратичная ошибка

$$Q(w) = \frac{1}{2m} \sum_{i=1}^m (f(x_i, w) - y_i)^2.$$

0.1) Регуляризация

Для уменьшения эффекта переобучения были реализованы несколько вариантов регуляризации:

- без регуляризации;
- L_1 -регуляризация;
- L_2 -регуляризация;
- Elastic Net регуляризация.

Во всех случаях минимизируется функция потерь вида

$$L(w) = Q(w) + R(w),$$

где $Q(w)$ представляет собой среднеквадратичную ошибку на обучающей выборке, а $R(w)$ является регуляризационным слагаемым.

Для L_1 -регуляризации используется выражение

$$R(w) = \lambda \sum_i |w_i|,$$

которое способствует получению разреженных решений и уменьшению влияния малозначимых признаков.

Для L_2 -регуляризации используется квадратичный штраф

$$R(w) = \lambda \sum_i w_i^2.$$

Данный подход ограничивает рост коэффициентов модели и позволяет повысить её устойчивость к шуму в данных.

Elastic Net объединяет преимущества двух предыдущих подходов и определяется выражением

$$R(w) = \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2.$$

В рамках программной реализации каждый тип регуляризации представлен отдельным классом, содержащим методы вычисления значения регуляризационного слагаемого и его градиента. Это позволяет использовать один и тот же код оптимизаторов для всех исследуемых вариантов функции потерь.

Итоговая функция потерь имеет вид

$$L(w) = Q(w) + R(w)$$

где $R(w)$ обозначает соответствующий регуляризационный член.

Реализованные методы оптимизации

Аналитическое решение

Для линейной регрессии сделано аналитическое решение (метод наименьших квадратов). Параметры модели вычисляются напрямую по обучающей выборке без выполнения итерационного процесса.

Данный подход используется в качестве базового метода сравнения, поскольку позволяет получить точное решение задачи линейной регрессии.

Стохастический градиентный спуск (SGD)

Стохастический градиентный спуск выполняет обновление параметров модели после обработки одного случайно выбранного объекта обучающей выборки. Обновление параметров выполняется по формуле

$$w_{k+1} = w_k - \eta \nabla L(w_k),$$

где η – шаг обучения, а $\nabla L(w_k)$ – градиент функции потерь.

Метод обладает низкой вычислительной стоимостью одной итерации, однако может демонстрировать высокую дисперсию траектории оптимизации.

Mini-Batch Gradient Descent

В отличие от SGD, градиент вычисляется по небольшой группе объектов (мини-пакету). Обновление параметров выполняется по той же формуле градиентного спуска, однако градиент вычисляется по нескольким наблюдениям одновременно.

Использование мини-пакетов позволяет уменьшить шум градиента и повысить устойчивость процесса обучения.

Метод Гаусса–Ньютона

Метод Гаусса–Ньютона относится к методам второго порядка и используется для решения задач нелинейной наименьших квадратов.

На каждой итерации решается система

$$(J^T J)p = -J^T r,$$

где J – матрица Якоби модели, а r – вектор невязок.

После вычисления направления p параметры обновляются по правилу

$$w_{k+1} = w_k + p.$$

Метод обычно обеспечивает более быструю сходимость по сравнению с градиентными методами.

Метод Левенберга–Марквардта

Метод Левенберга–Марквардта представляет собой модификацию метода Гаусса–Ньютона с дополнительным регуляризующим параметром.

На каждой итерации решается система

$$(J^T J + \lambda I)p = -J^T r,$$

где λ является параметром демпфирования.

При успешном шаге значение λ уменьшается, а при неудачном увеличивается. Благодаря этому метод сочетает преимущества метода Гаусса–Ньютона и градиентного спуска, обеспечивая устойчивую сходимость даже для сложных нелинейных задач.

Структура проекта

- **optimizers** – содержит реализации аналитического метода, SGD, mini-batch градиентного спуска, метода Гаусса–Ньютона и метода Левенберга–Марквардта
- **models** – содержит реализации регрессии и базовый интерфейс моделей
- **utils** – вспомогательные функции для вычисления метрик качества и построения графиков

- `losses.py` – функции потерь
- `regularization.py` – реализация различных видов регуляризации
- `main.py` – запуск экспериментов и формирование результатов

Эксперимент 1. Сравнение методов оптимизации

В рамках первого эксперимента исследуется работа различных методов оптимизации при решении задач регрессии. Рассматриваются две тестовые функции:

- слабонелинейная функция;
- нелинейная функция.

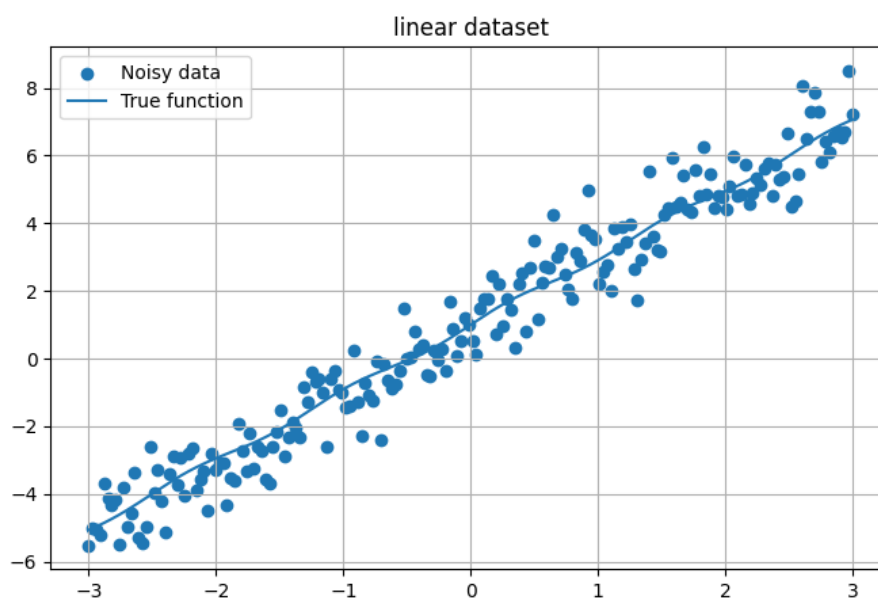
Для каждой функции генерируется обучающая выборка с добавлением гауссовского шума. В качестве моделей используются линейная регрессия и полиномиальная регрессия степеней от 2 до 5. На данном этапе регуляризация не применяется.

Для обучения используются следующие методы:

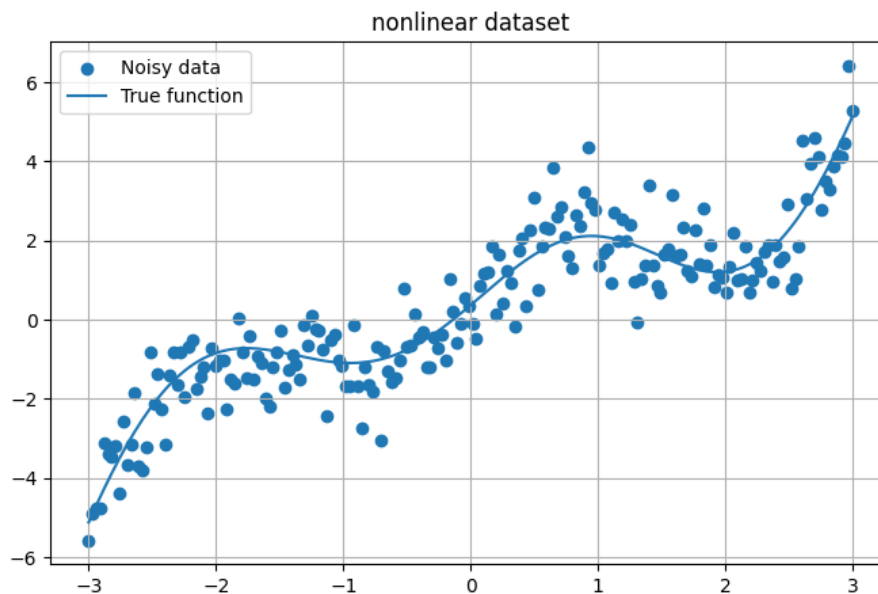
- аналитическое решение;
- SGD;
- Mini-Batch Gradient Descent;
- Gauss-Newton;
- Levenberg-Marquardt.

Исходные данные

Выборка, сформированная на основе слабонелинейной функции:



Выборка для нелинейной функции:



Результаты для слабонелинейной функции

Результаты аппроксимации слабонелинейной функции различными моделями и методами оптимизации:

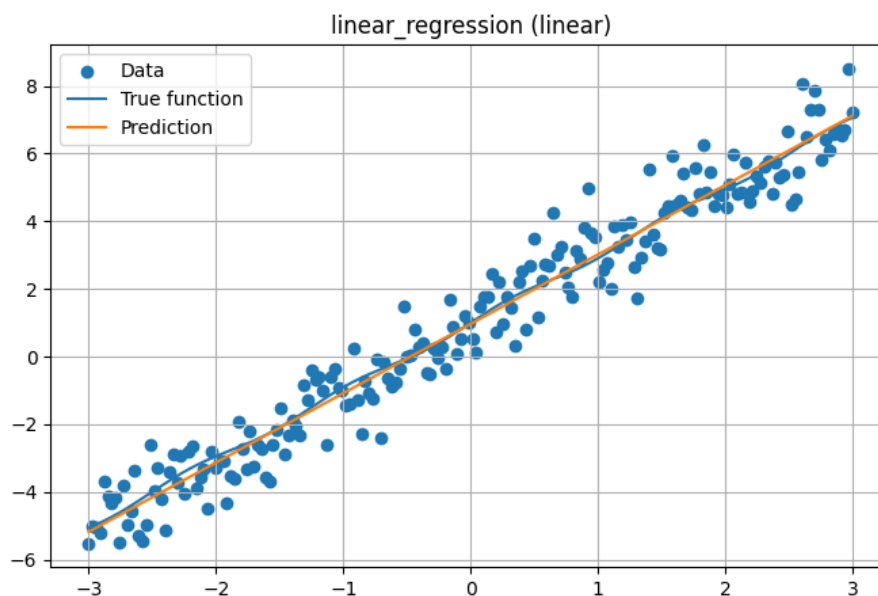


Рис. 1: Линейная регрессия

Как можно было догадаться, линейная модель сможет описать общий тренд данных, однако не воспроизведет нелинейные особенности исходной зависимости.

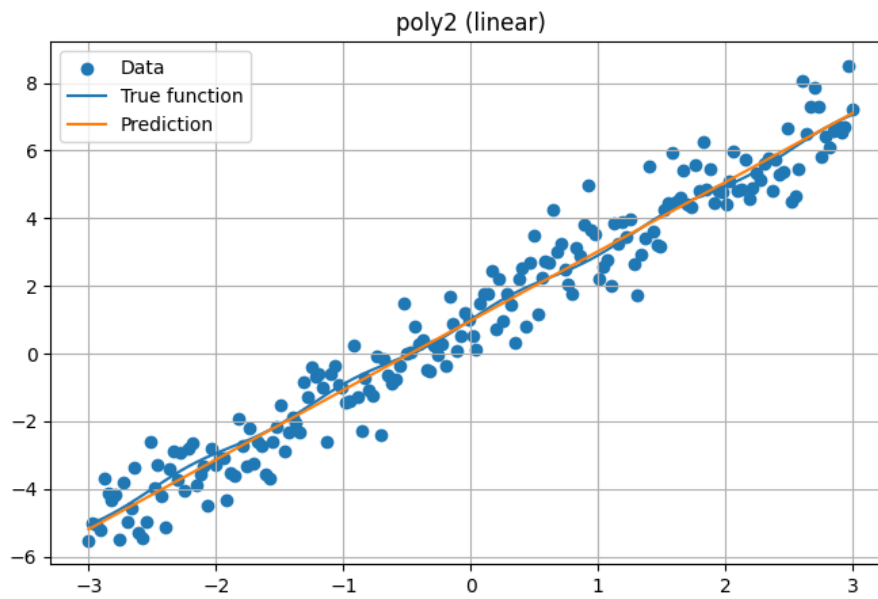


Рис. 2: Полином второй степени

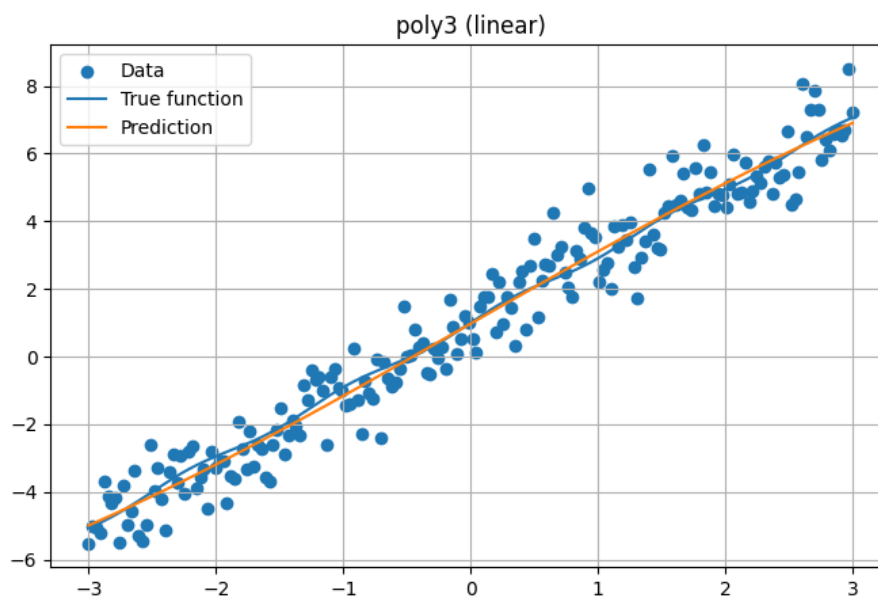


Рис. 3: Полином третьей степени

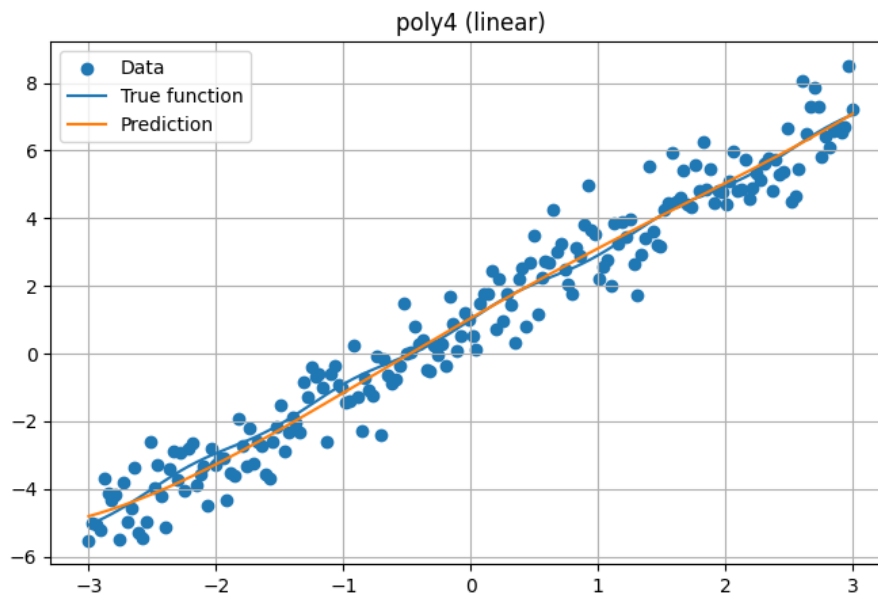


Рис. 4: Полином четвёртой степени

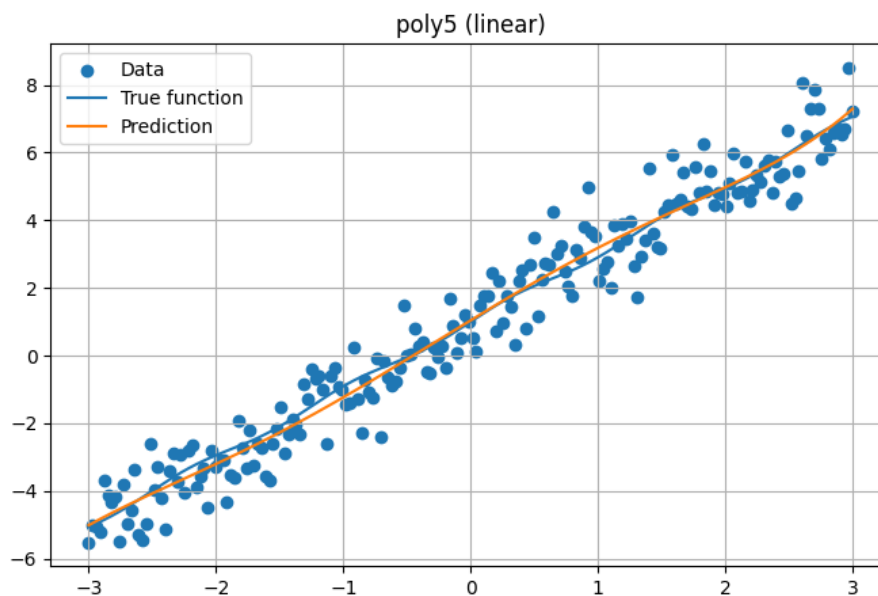


Рис. 5: Полином пятой степени

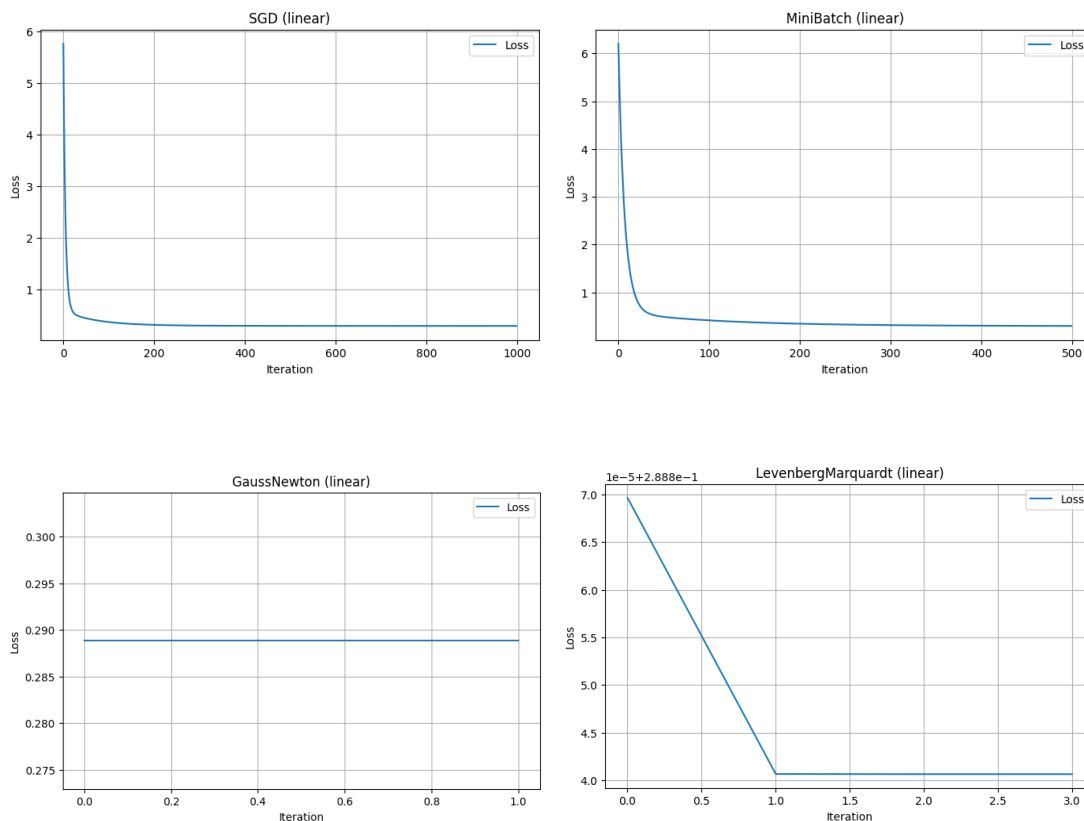
Таблица 1: Лучшие результаты для различных моделей

Модель	MSE	R^2
Линейная регрессия	0.593	0.956
Полиномиальная регрессия (2)	0.592	0.956
Полиномиальная регрессия (3)	0.586	0.956
Полиномиальная регрессия (4)	0.582	0.956
Полиномиальная регрессия (5)	0.578	0.957

С увеличением степени полинома качество аппроксимации должно возрастать,

поскольку модель получает возможность учитывать нелинейные особенности данных.

Сравнение методов обучения



Для SGD наблюдается плавное уменьшение значения функции потерь по мере увеличения числа эпох.

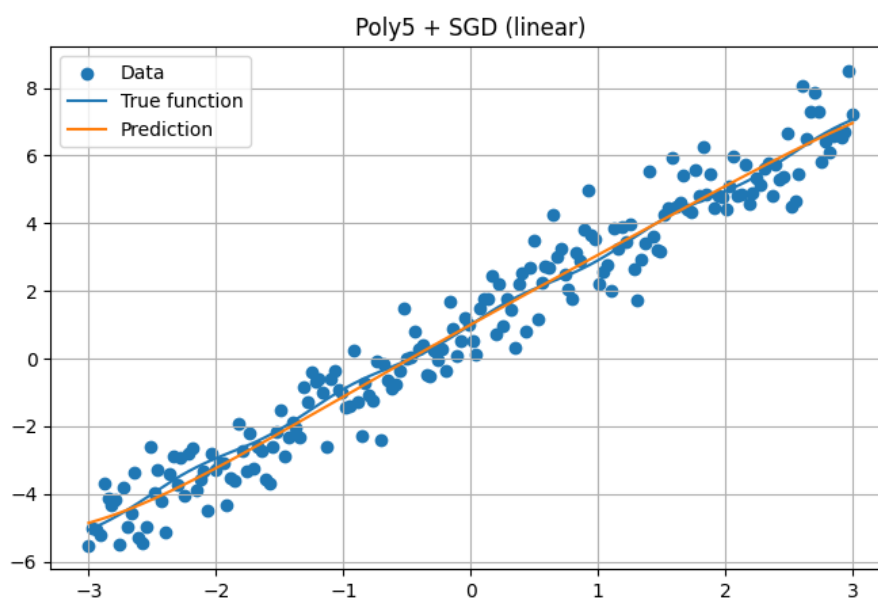
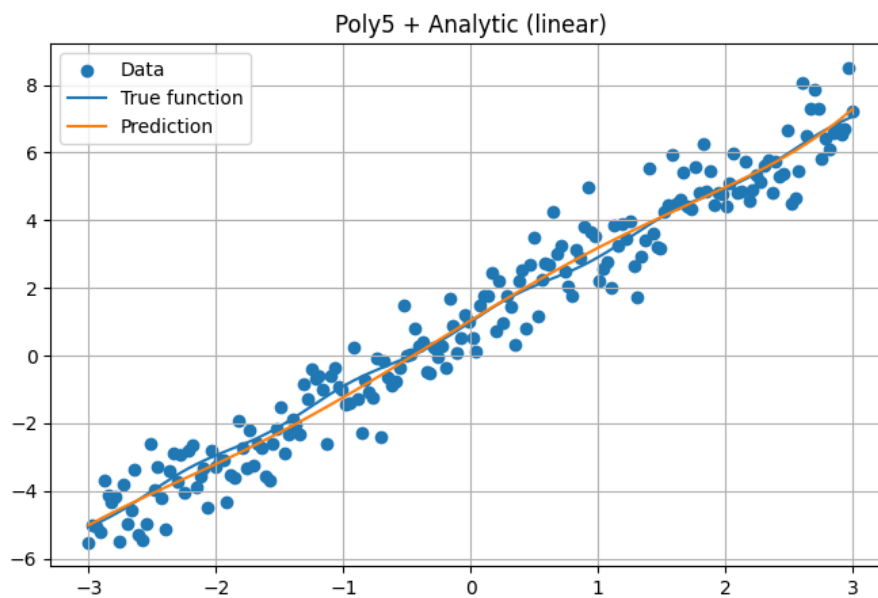
М-BGD демонстрирует более сглаженную динамику благодаря использованию мини-батчей фиксированного размера. Однако итоговое значение функции потерь оказалось выше по сравнению с другими методами.

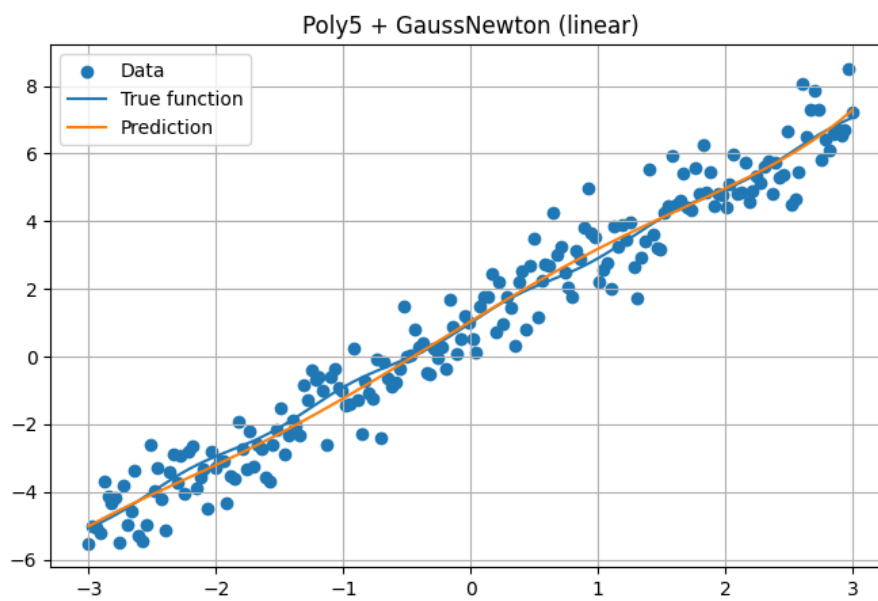
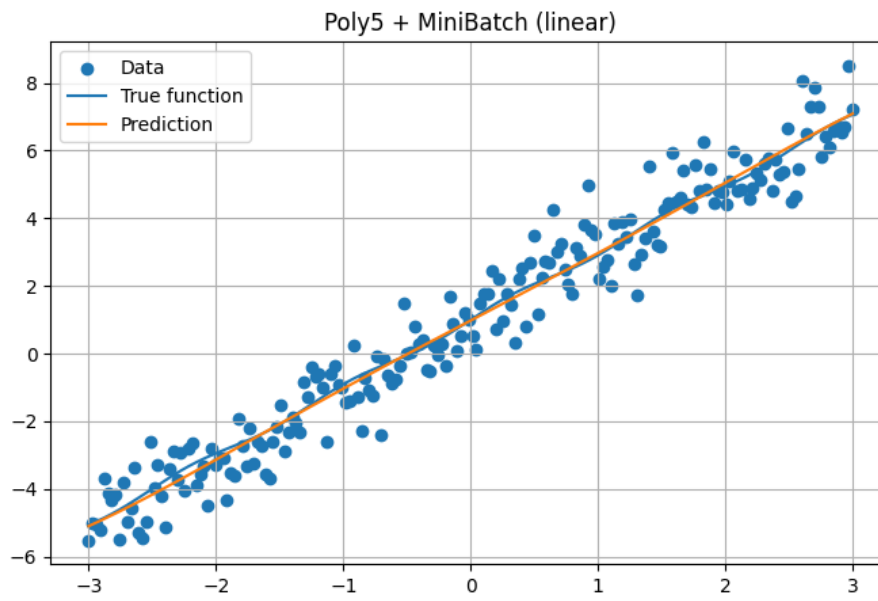
Метод Гаусса–Ньютона достигает минимума практически за одну-две итерации. Это связано с тем, что рассматриваемая задача является линейной по параметрам.

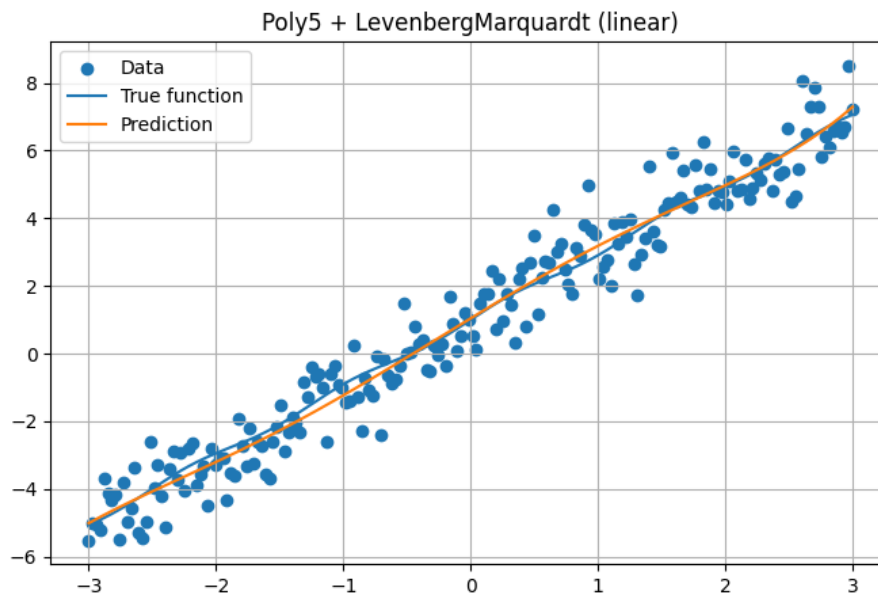
Алгоритм Левенберга–Марквардта также демонстрирует очень быструю сходимость и достигает решения, практически совпадающего с аналитическим.

Методы Гаусса–Ньютона и Левенберга–Марквардта достигали практически идентичного результата. Это связано с тем, что используемые модели являются линейными относительно параметров, а потому задача сводится к решению системы нормальных уравнений.

Сравнение методов оптимизации на одной модели







Полученные кривые практически совпадают. Все методы смогли найти близкое решение и качественно восстановить исходную зависимость.

Наиболее заметные отличия наблюдаются для SGD и Mini-Batch Gradient Descent, поскольку данные методы используют приближённые оценки градиента и останавливаются после конечного числа эпох. Тем не менее различия в итоговом качестве оказываются незначительными.

Результаты для нелинейной функции

Аналогичные эксперименты для более сложной нелинейной зависимости.

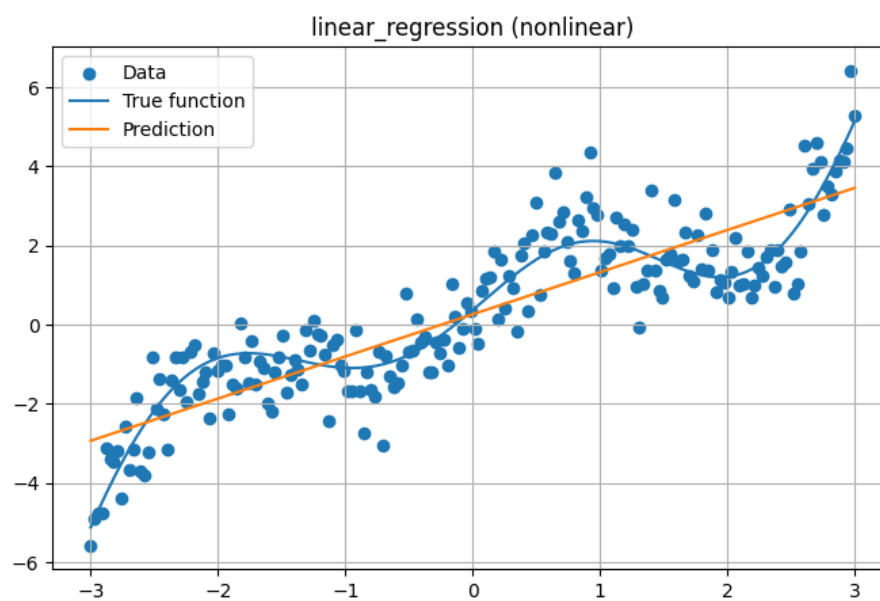


Рис. 6: Линейная регрессия

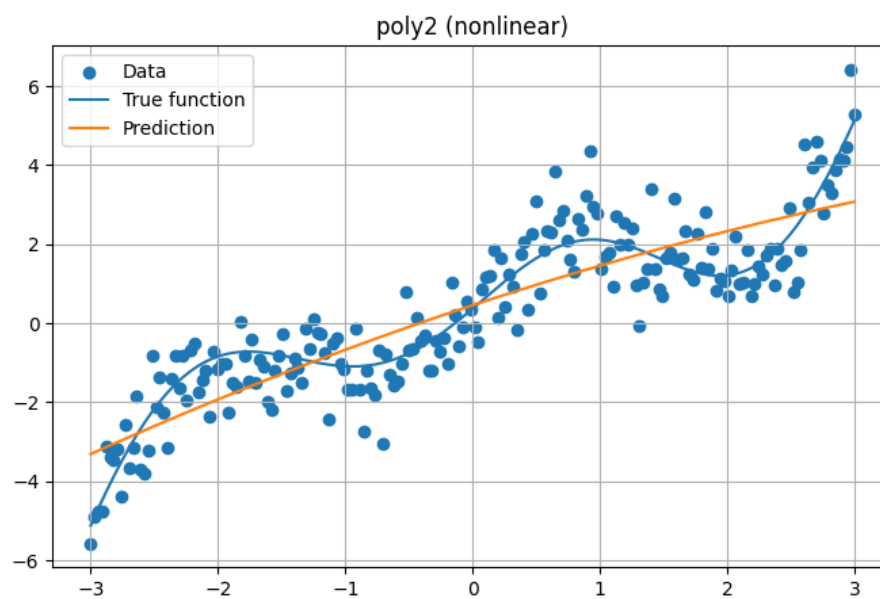


Рис. 7: Полином второй степени

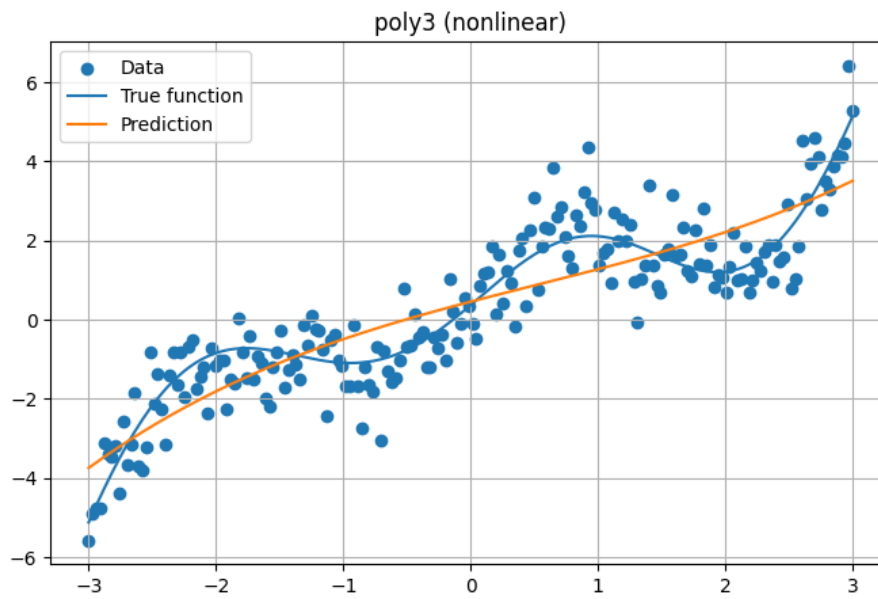


Рис. 8: Полином третьей степени

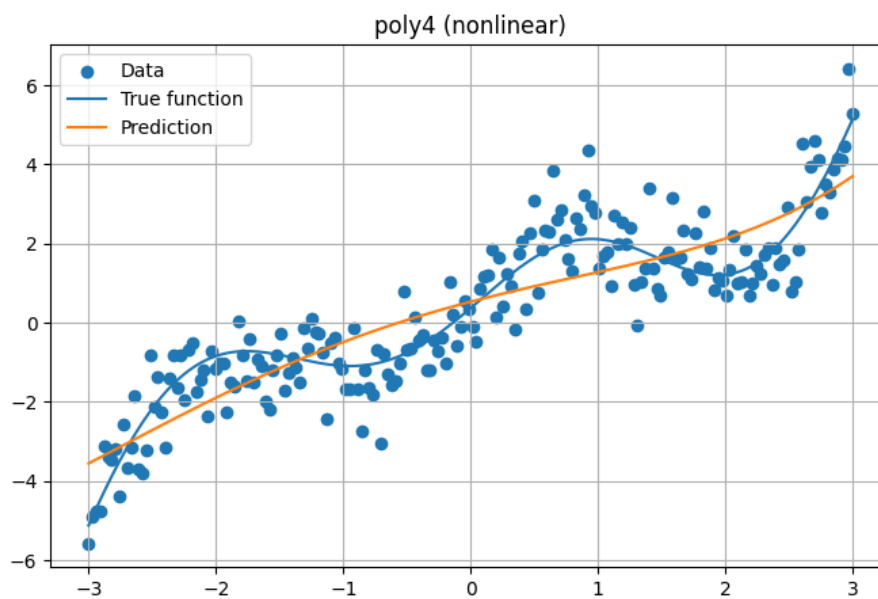


Рис. 9: Полином четвёртой степени

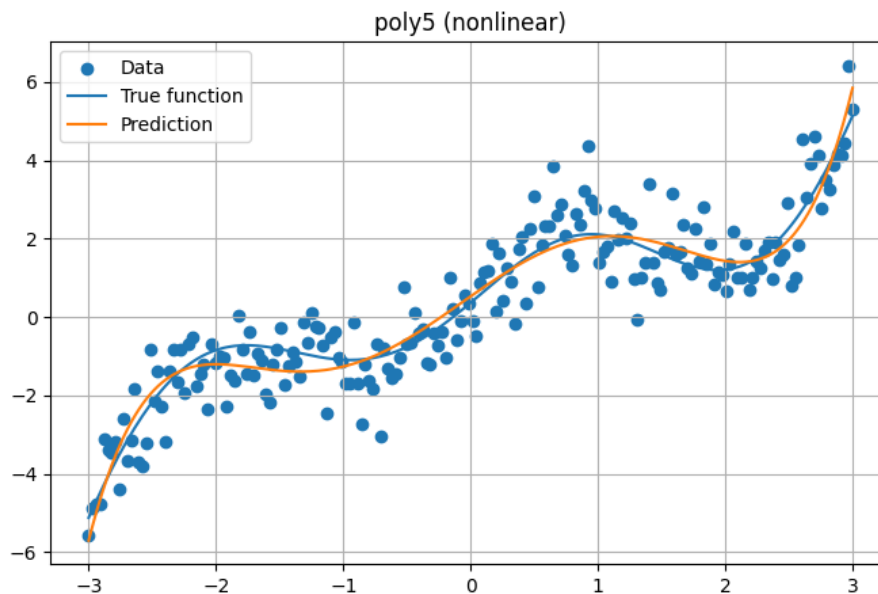
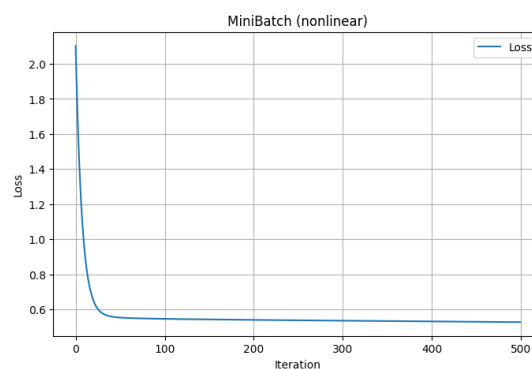
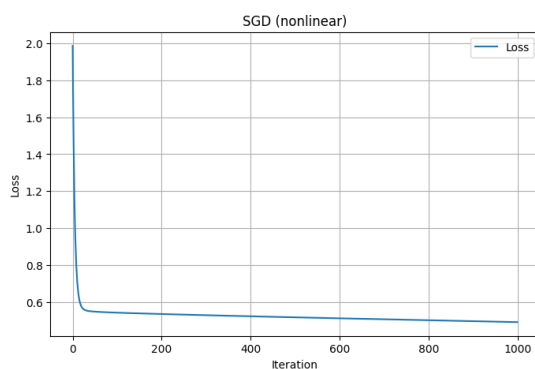


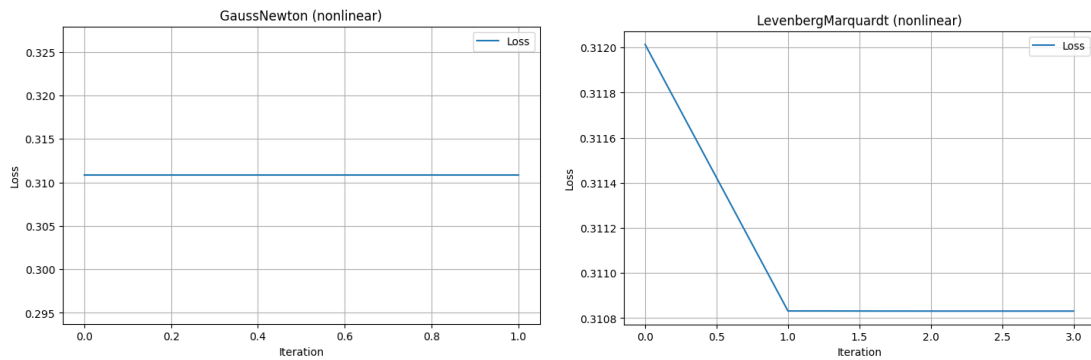
Рис. 10: Полином пятой степени

Модель	MSE	R^2
Линейная регрессия	1.172	0.746
Полиномиальная регрессия (2)	1.142	0.752
Полиномиальная регрессия (3)	1.114	0.758
Полиномиальная регрессия (4)	1.109	0.759
Полиномиальная регрессия (5)	0.622	0.865

Для нелинейной функции влияние степени полинома должно проявляться значительно сильнее. В нашем же примере так вышло, что функция плохо представима через полиномы степени меньше 5 и очень хорошо представима полиномом 5 степени, что связано напрямую с её формой.

Сравнение методов обучения





Для более сложной нелинейной зависимости различия между методами проявляются заметнее. Градиентные методы требуют значительно большего количества итераций для достижения приемлемого качества решения.

Метод Гаусса–Ньютона и алгоритм Левенберга–Марквардта по-прежнему быстро достигают минимума функции потерь благодаря использованию информации о локальной структуре задачи.

Наиболее медленной оказывается сходимость SGD, однако даже в этом случае наблюдается устойчивое уменьшение функции потерь на протяжении процесса обучения.

Сравнение методов оптимизации на одной модели

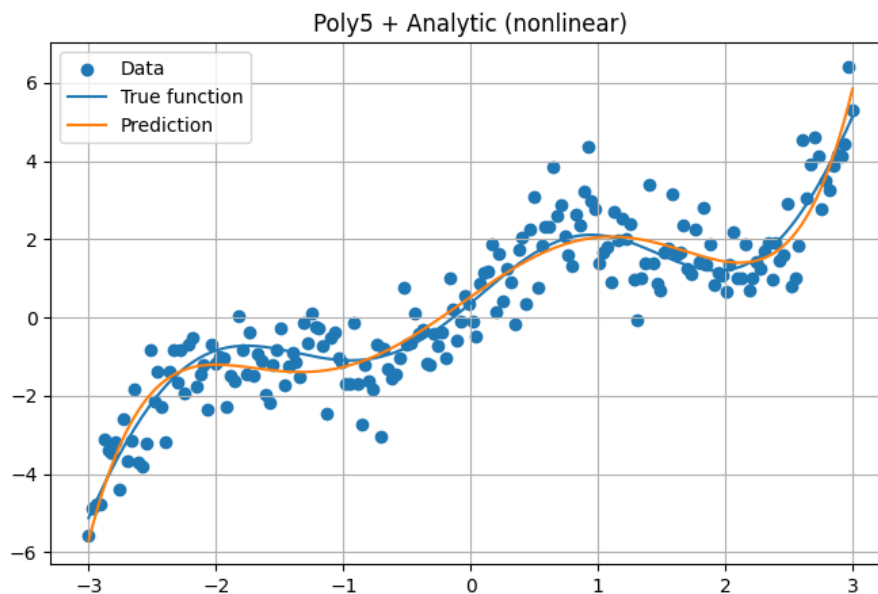


Рис. 11: Аналитическое решение

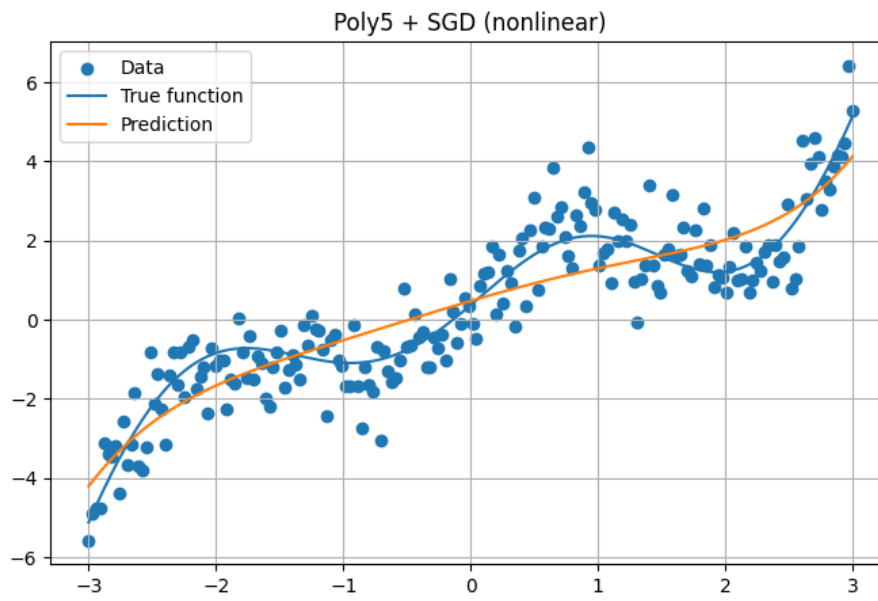


Рис. 12: SGD

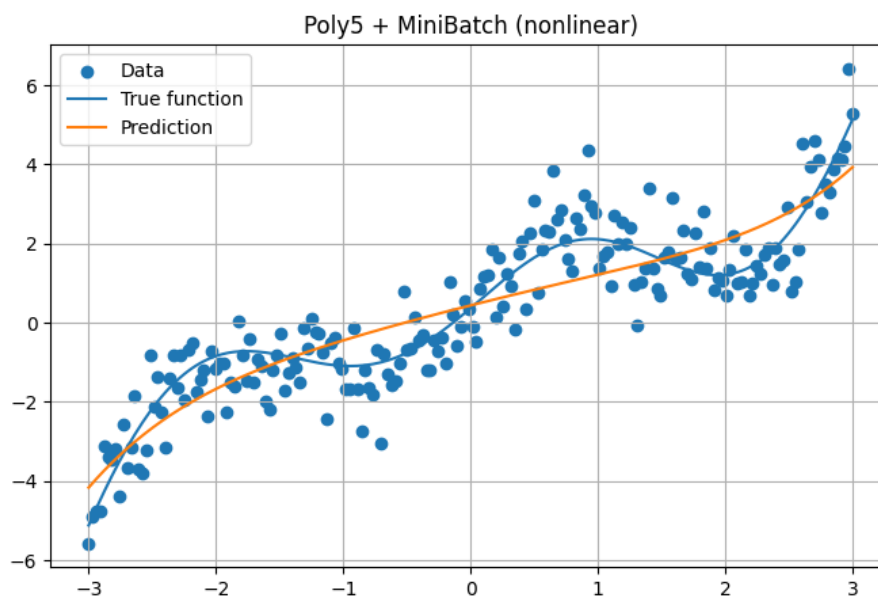


Рис. 13: Mini-Batch Gradient Descent

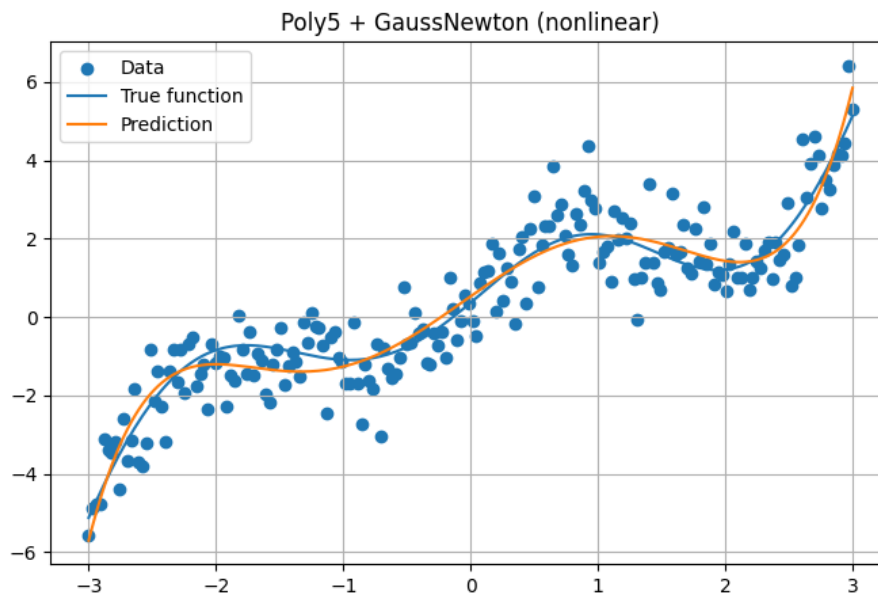


Рис. 14: Метод Гаусса–Ньютона

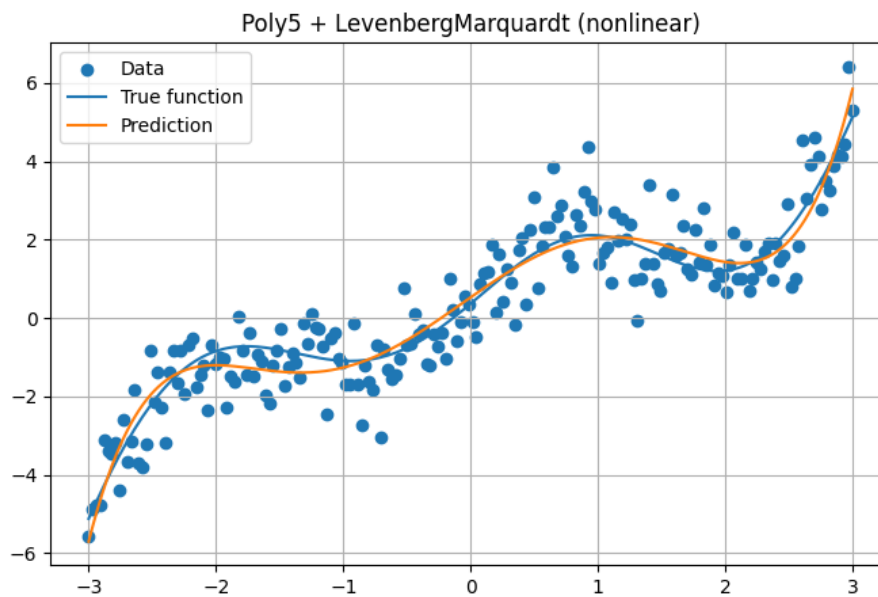


Рис. 15: Метод Левенберга–Марквардта

Методы второго порядка и аналитическое решение приводят к практически одинаковой регрессионной кривой. SGD и Mini-Batch Gradient Descent демонстрируют несколько худшее приближение, что согласуется с ранее полученными значениями функции потерь.

Несмотря на это, все методы корректно восстанавливают общий характер исследуемой зависимости.

Сравнение значений loss и качества аппроксимации

Для количественного сравнения методов обучения были рассчитаны итоговые значения функции потерь, а также основные метрика коэффициент детерминации R^2 .

Для каждой тестовой функции дополнительно построены сравнительные диаграммы, отражающие итоговые значения метрик для различных сочетаний модели и метода оптимизации.

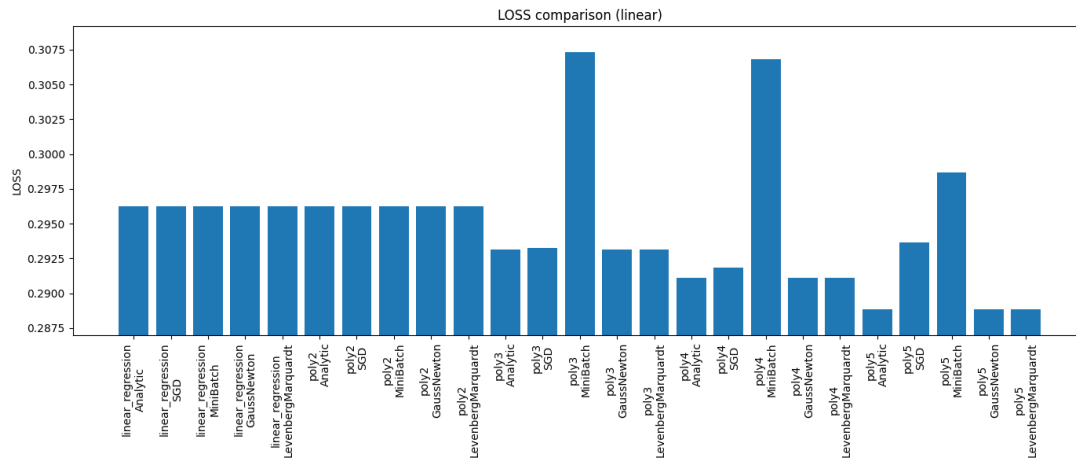


Рис. 16: loss для слабонелинейной функции

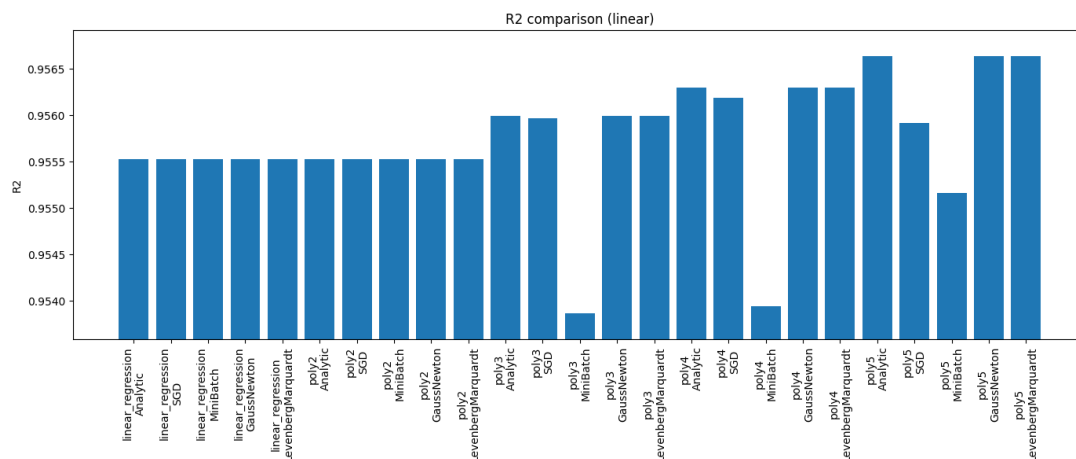


Рис. 17: R^2 для слабонелинейной функции

Из графиков видно, что для слабонелинейной зависимости различия между методами оптимизации практически отсутствуют. Все методы сходятся к одному и тому же решению, а качество аппроксимации определяется в первую очередь выбором модели.

Наилучший результат демонстрирует полином пятой степени, обеспечивающий минимальное значение функции потерь и максимальный коэффициент детерминации.

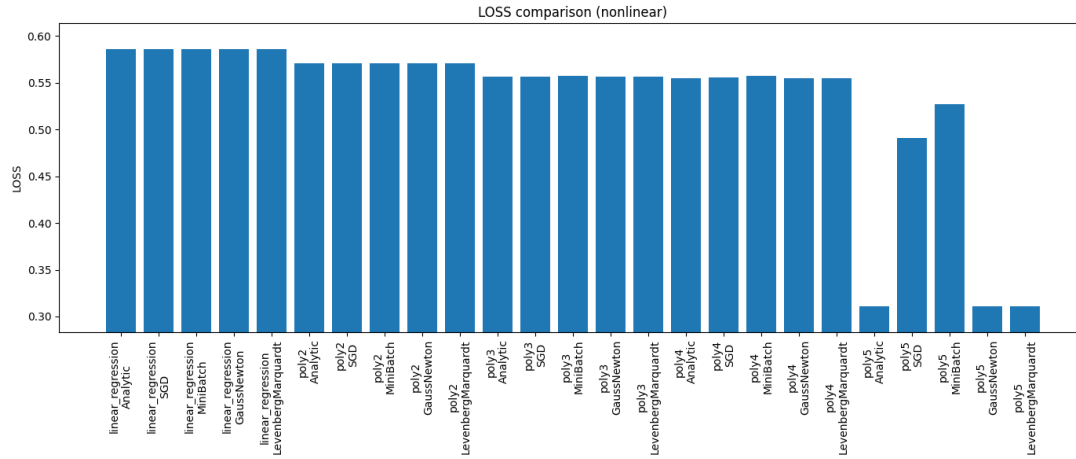


Рис. 18: loss для нелинейной функции

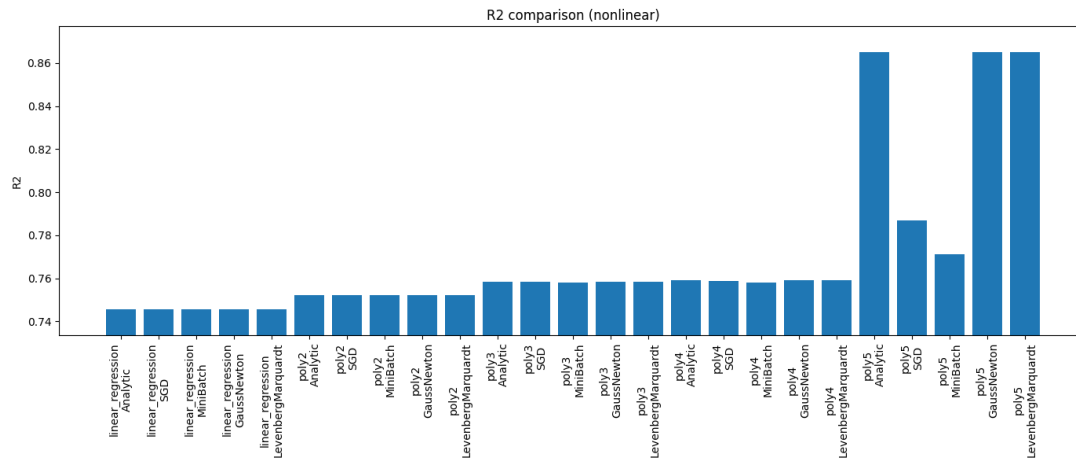


Рис. 19: R^2 для нелинейной функции

Для нелинейной функции влияние выбора модели оказывается существенно сильнее. Полиномы малых степеней не способны корректно воспроизвести структуру исходной зависимости, вследствие чего значения ошибки остаются высокими.

Полиномиальная регрессия пятой степени обеспечивает заметное улучшение качества аппроксимации и достигает значения $R^2 \approx 0.865$, что существенно превосходит результаты остальных моделей.

При этом различия между аналитическим решением, методом Гаусса–Ньютона и алгоритмом Левенберга–Марквардта практически отсутствуют, поскольку рассматриваемая задача является линейной по параметрам.

Таблица 2: Средние характеристики методов оптимизации

Метод	Средний loss	Среднее число итераций	Среднее время, с
Analytic	0.4045	1.0	0.0365
SGD	0.4232	1000.0	1.9604
MiniBatch	0.4970	500.0	0.1772
GaussNewton	0.4045	2.0	0.0831
LevenbergMarquardt	0.4045	5.6	0.0794

Аналитическое решение обеспечивает минимальное значение функции потерь за один шаг и является наиболее эффективным методом при наличии замкнутой формулы решения.

Метод Гаусса–Ньютона достигает того же качества решения всего за две итерации. Метод Левенберга–Марквардта также сходится к тому же минимуму, однако требует немного большего числа итераций из-за адаптивного выбора параметра регуляризации.

SGD и Mini-Batch Gradient Descent демонстрируют существенно большее число итераций. При этом SGD достигает качества, близкого к оптимальному, тогда как Mini-Batch Gradient Descent в используемой конфигурации не успевает полностью сойтись за отведённое число эпох и показывает большее итоговое значение функции потерь.

Таким образом, для рассматриваемых задач наилучшие результаты по скорости сходимости демонстрируют аналитическое решение и методы второго порядка, тогда как градиентные методы требуют значительно большего числа обновлений параметров.

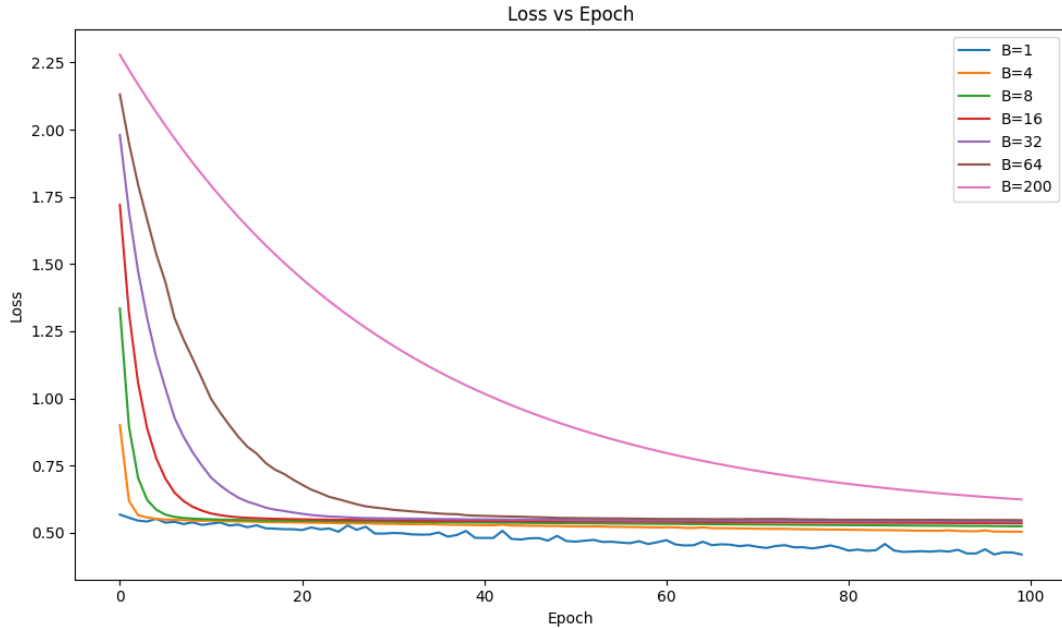
Эксперимент 2. Исследование влияния размера mini-batch

Для исследования была выбрана нелинейная задача регрессии и полиномиальная модель 5 степени. Обучение выполнялось методом Mini-Batch Gradient Descent при различных размерах пакета:

$$B \in \{1, 4, 8, 16, 32, 64, 200\}.$$

Для каждого значения B были построены графики изменения функции потерь по эпохам и по числу вычислений градиента, но в отчёт вошли только общие графики сравнения, потому что, по нашему мнению, они и так хорошо иллюстрируют как каждый график в отдельности, так и их сравнение и не перегружают отчет.

Динамика по эпохам



На рисунке видно существенное влияние размера пакета на характер сходимости метода. Для малых значений batch ($B = 1, 4, 8$) функция потерь уменьшается наиболее быстро по эпохам. Это объясняется тем, что за одну эпоху выполняется большое количество обновлений параметров, благодаря чему алгоритм быстрее адаптируется к структуре данных.

По мере увеличения размера пакета ($B = 16, 32, 64$) кривые становятся более гладкими и стабильными. Градиент вычисляется по большему числу объектов, поэтому его оценка оказывается менее шумной. Вместе с этим уменьшается количество обновлений параметров за эпоху, что приводит к более медленному снижению функции потерь.

Для полного пакета ($B = m$) наблюдается наиболее гладкая траектория обучения. В этом случае каждая эпоха соответствует единственному вычислению градиента по всей выборке. Несмотря на высокую стабильность, скорость уменьшения функции потерь по эпохам оказывается наименьшей, поскольку обновления параметров происходят существенно реже.

Таким образом, график по эпохам демонстрирует классический компромисс между скоростью обучения и устойчивостью траектории: малые batch обеспечивают быстрое уменьшение ошибки, но сопровождаются заметными колебаниями, тогда как большие batch дают более плавную, но менее агрессивную динамику сходимости.

Динамика по числу вычислений градиента

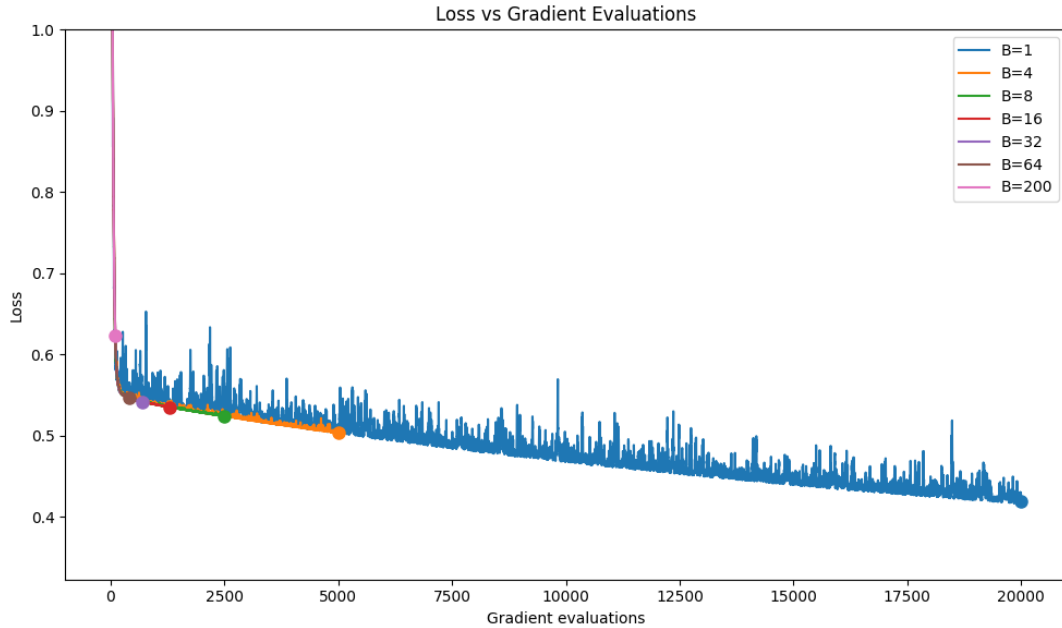


График зависимости функции потерь от числа вычислений градиента позволяет сравнить различные размеры пакета с точки зрения вычислительных затрат. В отличие от предыдущего графика, здесь по оси абсцисс отложено фактическое количество вычислений градиента, поэтому сравнение не зависит от того, сколько обновлений параметров выполняется за одну эпоху.

Результаты подтверждают выводы, сделанные при анализе графика по эпохам, однако позволяют посмотреть на процесс обучения с другой стороны. Если на предыдущем графике малые значения batch выглядели наиболее быстрыми, то после учёта количества вычислений градиента их преимущество становится менее выраженным.

Можно заметить, что размеры пакета $B = 4$ и $B = 8$ обеспечивают наиболее быстрое уменьшение функции потерь относительно затраченных вычислительных ресурсов. Для $B = 1$ наблюдается высокая частота обновлений параметров, однако это требует значительно большего количества вычислений градиента. В свою очередь, большие значения B демонстрируют более предсказуемое поведение, но для достижения сопоставимого качества решения требуют большего числа вычислений на каждом шаге оптимизации.

Таким образом, сравнение по числу вычислений градиента показывает, что наиболее эффективными для рассматриваемой задачи являются промежуточные размеры пакета. Они сохраняют преимущества mini-batch подхода, обеспечивая достаточно быстрое снижение функции потерь без существенного роста вычислительных затрат.

Вывод

Проведённый эксперимент показал, что размер mini-batch оказывает существенное влияние как на скорость обучения, так и на вычислительную эффективность алгоритма.

Малые размеры пакета обеспечивают наиболее быстрое уменьшение функции потерь по эпохам, однако сопровождаются более высоким числом вычислений гради-

ента. Большие размеры пакета делают процесс обучения более стабильным, но снижают частоту обновления параметров и замедляют сходимость.

Для рассматриваемой нелинейной задачи и полиномиальной регрессии пятой степени наиболее удачными оказались размеры пакета $B = 4$ и $B = 8$. Они обеспечивают хороший баланс между скоростью сходимости, устойчивостью процесса обучения и вычислительной эффективностью, достигая низких значений функции потерь при умеренных затратах вычислительных ресурсов.

Эксперимент 3. Исследование регуляризации

Цель эксперимента — исследовать влияние различных методов регуляризации на процесс обучения регрессионной модели и её способность бороться с переобучением.

Для эксперимента была выбрана нелинейная задача регрессии, использовавшаяся в предыдущих разделах работы:

$$f(x) = \sin(2x) + 0.2x^3 + \exp(-(x - 1)^2).$$

В качестве модели использовалась полиномиальная регрессия высокой степени. Полиномы высокой степени обладают большой выразительной способностью и способны подстраиваться под шум в данных, что делает проблему переобучения особенно заметной. Поэтому такая модель является удобным объектом для исследования методов регуляризации.

Рассматривались три популярных подхода к регуляризации: L1, L2, Elastic Net.

Для каждого метода были исследованы несколько значений коэффициентов регуляризации. В ходе эксперимента анализировалась динамика следующих величин:

- полный функционал потерь;
- эмпирический риск;
- вклад регуляризационного слагаемого.

Такое представление позволяет отдельно оценить влияние регуляризации на качество аппроксимации данных и на ограничение сложности модели.

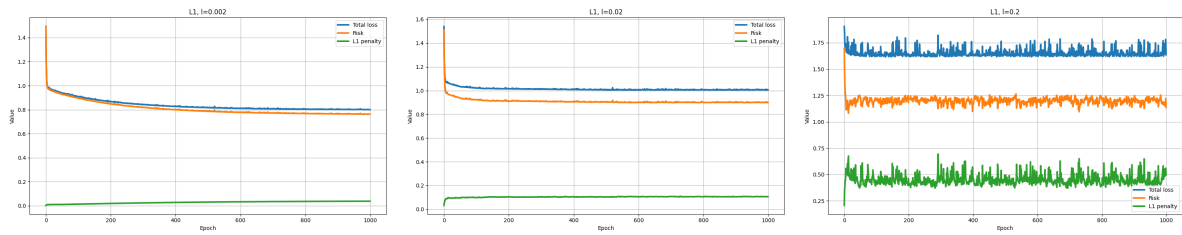
Динамика функционала потерь

Для каждого метода регуляризации были рассмотрены три значения коэффициента регуляризации:

$$\lambda \in \{0.001, 0.01, 0.1\}.$$

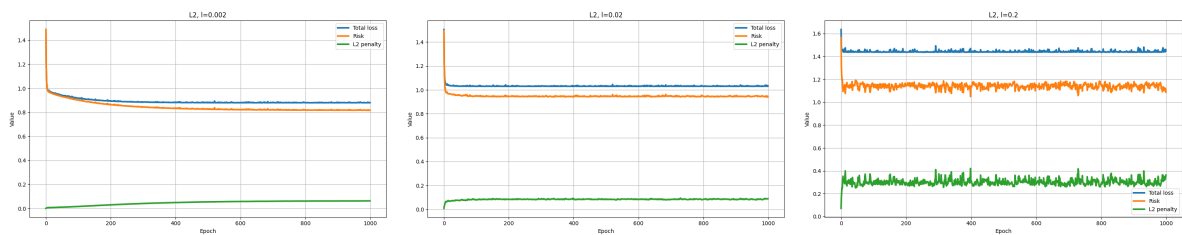
На графиках представлены динамика полного функционала потерь, эмпирического риска и соответствующих регуляризационных слагаемых в процессе обучения.

L1-регуляризация



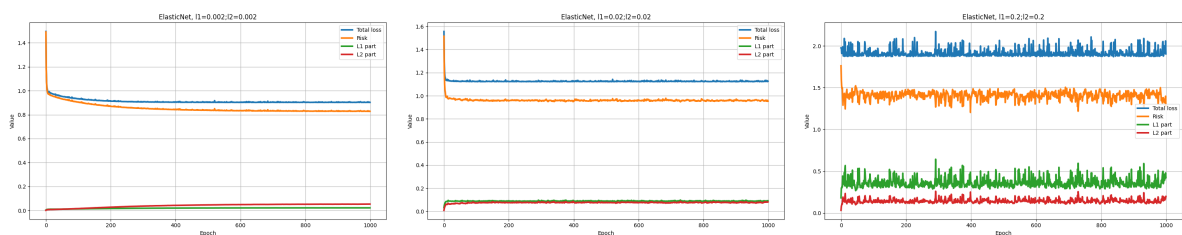
Для малых значений коэффициента регуляризации ($\lambda = 0.001$) вклад L1-слагаемого практически не влияет на процесс обучения, поэтому полный функционал потерь близок к эмпирическому риску. При увеличении коэффициента регуляризации становится заметным рост вклада штрафа за величину коэффициентов модели. Для $\lambda = 0.1$ регуляризационное слагаемое составляет существенную часть полного функционала потерь.

L2-регуляризация



Для L2-регуляризации наблюдается аналогичная тенденция.

Elastic Net



Elastic Net объединяет свойства L1 и L2-регуляризации. При увеличении коэффициентов регуляризации возрастает суммарный вклад штрафа, а эмпирический риск снижается менее агрессивно. Полученные графики демонстрируют ожидаемое промежуточное поведение между L1 и L2-подходами.

Таким образом, увеличение коэффициента регуляризации приводит к росту вклада регуляризационных слагаемых в полный функционал потерь. При малых значениях λ обучение практически совпадает с обучением без регуляризации, тогда как при больших значениях регуляризация начинает существенно влиять на процесс оптимизации и ограничивать сложность модели.

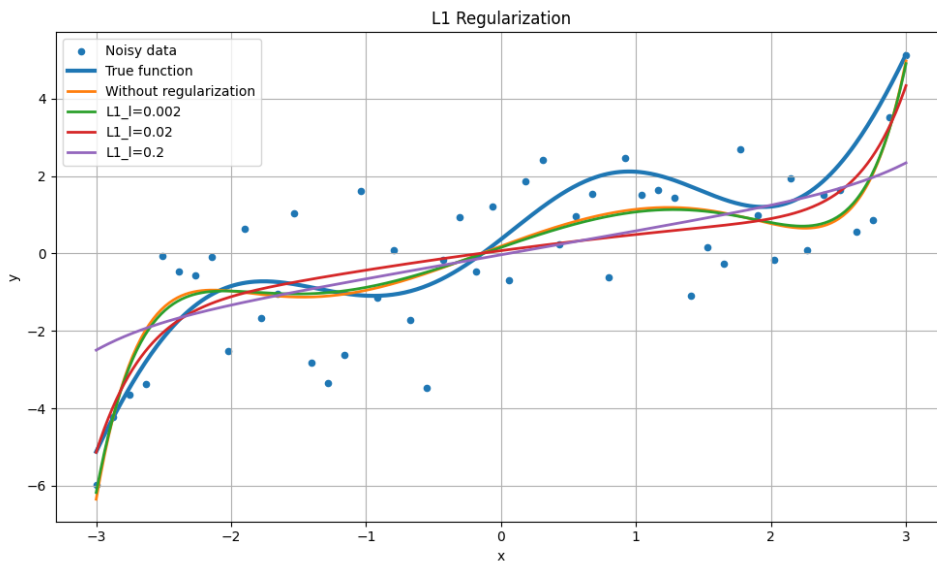
Сравнение предсказаний моделей до и после регуляризации

Для дополнительной оценки влияния регуляризации были построены графики восстановления исходной зависимости. На каждом графике представлены:

- истинная зависимость $f(x)$;
- зашумленные обучающие точки;
- предсказания полиномиальной модели степени 10 без регуляризации;
- предсказания моделей с различными параметрами регуляризации.

Для демонстрации эффекта переобучения использовалась небольшая обучающая выборка из 50 точек с увеличенным уровнем шума $\sigma = 1.5$. Полином высокой степени в такой постановке обладает большой свободой и может подстраиваться под шумовые компоненты данных.

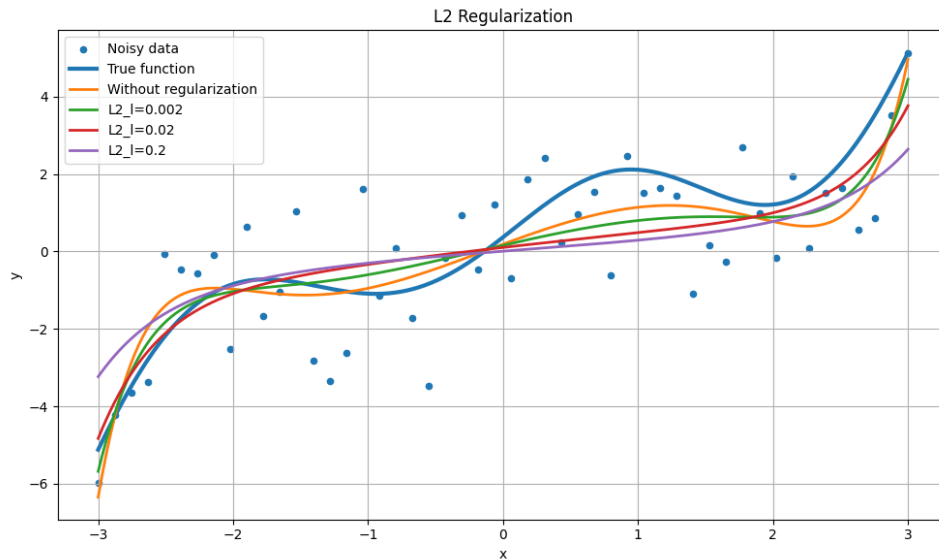
L1-регуляризация



На графике видно, что модель без регуляризации имеет сложную форму и пытается максимально точно описать обучающие точки. При этом возникают дополнительные колебания, не соответствующие истинной зависимости, что является признаком переобучения.

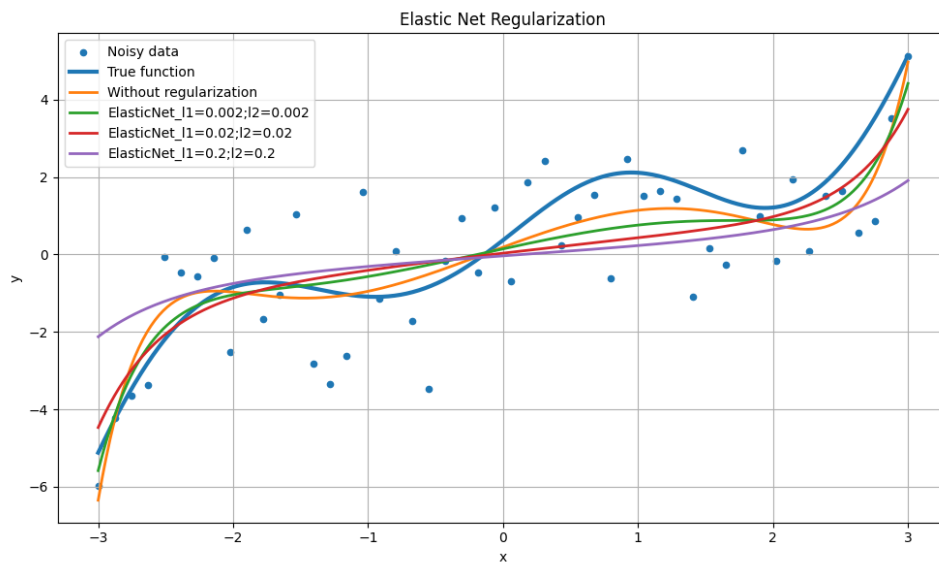
Добавление L1-регуляризации приводит к уменьшению отдельных коэффициентов полинома. При увеличении параметра λ модель становится менее сложной, а предсказанная зависимость становится более гладкой.

L2-регуляризация



L2-регуляризация ограничивает величину всех коэффициентов модели, поэтому изменение формы кривой происходит более плавно по сравнению с L1-подходом.

Elastic Net



Elastic Net объединяет свойства L1 и L2-регуляризации. В результате модель одновременно ограничивает размер коэффициентов и допускает уменьшение части коэффициентов до малых значений.

Общий вывод

Сравнение графиков показывает, что отсутствие регуляризации приводит к подгонке под обучающую выборку и появлению лишних колебаний полиномиальной функции.

Все рассмотренные методы регуляризации уменьшают сложность модели:

- L1-регуляризация сильнее влияет на структуру коэффициентов и способствует получению более разреженного решения;
- L2-регуляризация равномерно уменьшает коэффициенты модели и обеспечивает более плавную аппроксимацию;
- Elastic Net сочетает преимущества обоих методов.

Таким образом, применение регуляризации позволяет уменьшить переобучение модели высокой степени и улучшить восстановление истинной зависимости по зашумленным данным.

Сравнение коэффициентов моделей

Для количественной оценки влияния регуляризации были проанализированы значения коэффициентов полиномиальной модели степени 10. В таблице представлены абсолютные значения коэффициентов без регуляризации и после применения различных методов регуляризации.

Таблица 3: L1

Модель	$ w_0 $	$ w_1 $	$ w_2 $	$ w_3 $	$ w_4 $	$ w_5 $	$ w_6 $	$ w_7 $	$ w_8 $	$ w_9 $	$ w_{10} $
Без рег.	0.189	3.865	0.937	6.477	0.821	1.007	0.154	3.247	0.494	6.032	0.114
L1 ($\lambda = 0.002$)	0.157	3.582	0.529	5.950	0.006	0.259	0.004	2.464	0.266	5.701	0.001
L1 ($\lambda = 0.02$)	0.073	1.381	0.390	0.006	0.024	0.010	0.029	0.016	0.017	3.333	0.015
L1 ($\lambda = 0.2$)	0.035	1.847	0.009	0.092	0.021	0.132	0.006	0.166	0.004	0.182	0.004

Из таблицы видно, что L1-регуляризация активно зануляет коэффициенты модели. Уже при $\lambda = 0.02$ большинство коэффициентов становятся близкими к нулю, а при $\lambda = 0.2$ существенными остаются лишь несколько коэффициентов низких порядков. Это соответствует теоретическому свойству L1-регуляризации формировать разреженные решения и автоматически выполнять отбор признаков.

Таблица 4: L2

Модель	$ w_0 $	$ w_1 $	$ w_2 $	$ w_3 $	$ w_4 $	$ w_5 $	$ w_6 $	$ w_7 $	$ w_8 $	$ w_9 $	$ w_{10} $
Без рег.	0.189	3.865	0.937	6.477	0.821	1.007	0.154	3.247	0.494	6.032	0.114
L2 ($\lambda = 0.002$)	0.158	2.415	0.561	2.841	0.180	0.170	0.114	2.068	0.213	3.591	0.068
L2 ($\lambda = 0.02$)	0.103	1.194	0.248	0.251	0.121	0.638	0.113	0.987	0.095	1.227	0.059
L2 ($\lambda = 0.2$)	0.005	0.817	0.092	0.573	0.070	0.532	0.058	0.514	0.047	0.500	0.036

В отличие от L1-подхода, L2-регуляризация уменьшает все коэффициенты более равномерно. Даже при большом значении параметра регуляризации коэффициенты не зануляются полностью, а лишь уменьшаются по модулю. Благодаря этому модель сохраняет вклад большинства признаков, но становится менее чувствительной к шуму в обучающих данных.

Таблица 5: Elastic Net

Модель	$ w_0 $	$ w_1 $	$ w_2 $	$ w_3 $	$ w_4 $	$ w_5 $	$ w_6 $	$ w_7 $	$ w_8 $	$ w_9 $	$ w_{10} $
Без рег.	0.189	3.865	0.937	6.477	0.821	1.007	0.154	3.247	0.494	6.032	0.114
EN ($\lambda_1 = \lambda_2 = 0.002$)	0.146	2.267	0.467	2.494	0.001	0.013	0.074	1.789	0.170	3.424	0.018
EN ($\lambda_1 = \lambda_2 = 0.02$)	0.033	1.224	0.192	0.253	0.092	0.599	0.061	0.912	0.028	1.122	0.020
EN ($\lambda_1 = \lambda_2 = 0.2$)	0.038	0.757	0.028	0.422	0.017	0.328	0.018	0.275	0.006	0.235	0.002

Elastic Net демонстрирует промежуточное поведение между L1 и L2-регуляризацией. С одной стороны, часть коэффициентов становится практически нулевой, как в случае L1-регуляризации. С другой стороны, остальные коэффициенты уменьшаются более плавно, сохраняя вклад различных степеней полинома. Такой результат соответствует идее Elastic Net как комбинации двух типов регуляризации.

Эксперимент 4. Сравнение методов оптимизации

Использовалась нелинейная задача. Рассматривались те же методы оптимизации: SGD, MiniBatch GD, метод Гаусса–Ньютона, метод Левенберга–Марквардта.

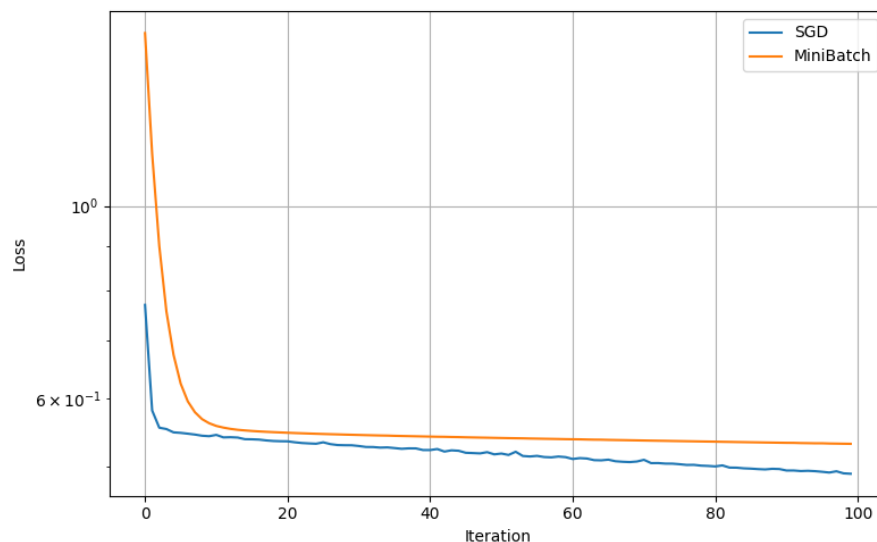
Для каждого метода измерялись итоговое значение функционала потерь, число итераций и время работы.

Сравнение основных характеристик

Результаты эксперимента представлены в таблице.

Метод	Loss	Итерации	Время, с
SGD ($lr = 0.01$)	0.4908	100	0.1114
MiniBatch GD ($lr = 0.01, batch = 4$)	0.5315	100	0.0638
Gauss–Newton	0.3108	2	0.00017
Levenberg–Marquardt	0.3108	4	0.00022

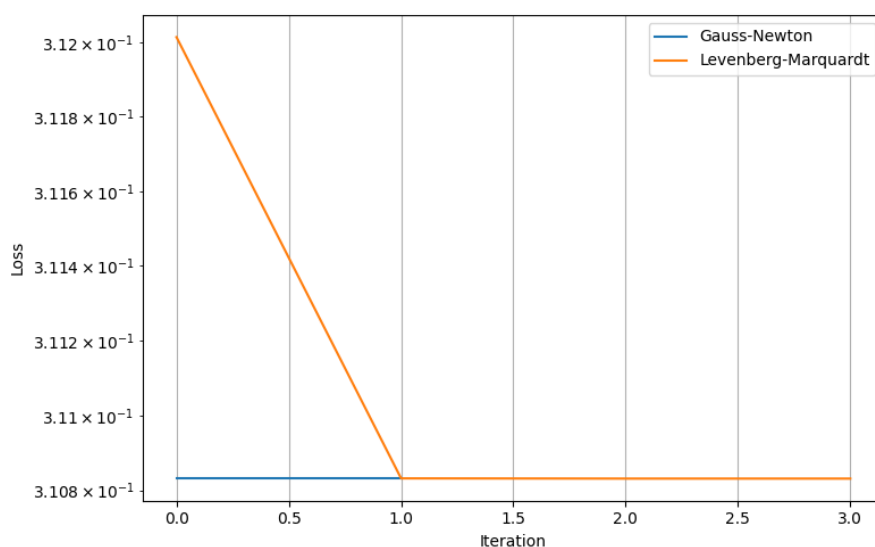
Полученные результаты показывают существенное преимущество методов второго порядка на рассматриваемой задаче. Методы Гаусса–Ньютона и Левенберга–Марквардта достигли меньшего значения функционала потерь по сравнению со стохастическими методами и потребовали значительно меньше итераций.



SGD и MiniBatch GD демонстрируют более медленную сходимость, поскольку используют информацию только о градиенте функции. Также следует отметить, что функции вообще не подошли к точности в $10e-8$, расчеты были прерваны спустя 100 эпох. При этом можно заметить на графике хорошую динамику SGD и плохую MiniBatch.

Сравнение Гаусса–Ньютона и Левенберга–Марквардта

Для более детального анализа была построена динамика значения функционала потерь для методов Гаусса–Ньютона и Левенберга–Марквардта.



Оба метода сошлись к одному и тому же значению $Loss \approx 0.3108$.

Для более детального анализа сходимости была сохранена история изменения функционала потерь для методов Гаусса–Ньютона и Левенберга–Марквардта.

Итерация	Gauss–Newton	Levenberg–Marquardt
1	0.310831	0.312013
2	0.310831	0.310832
3	—	0.310831
4	—	0.310831

Из таблицы видно, что метод Гаусса–Ньютона достигает итогового решения уже после первой итерации. На второй итерации значение функционала потерь не изменяется в пределах вычислительной точности, после чего алгоритм завершается.

Метод Левенберга–Марквардта также быстро сходится к тому же решению, однако делает несколько дополнительных итераций. После второй итерации значение функционала потерь отличается от оптимального менее чем на 10^{-6} , а последующие шаги лишь незначительно уточняют решение. Такое поведение связано с использованием регуляризованной матрицы Гессе и более осторожным выбором шага, что повышает устойчивость метода на сложных задачах ценой небольшого увеличения числа итераций.

Чувствительность к начальному приближению

Для оценки устойчивости методов были проведены запуски из различных начальных точек: нулевого вектора, вектора из единиц и случайного начального приближения.

Таблица 6: Влияние начального приближения

Метод	Начальная точка	Итерации	Итоговый Loss
Gauss–Newton	zeros	2	0.3108
Gauss–Newton	ones	2	0.3108
Gauss–Newton	random	2	0.3108
Levenberg–Marquardt	zeros	4	0.3108
Levenberg–Marquardt	ones	4	0.3108
Levenberg–Marquardt	random	4	0.3108

Во всех рассмотренных случаях оба метода сходились к одному и тому же решению независимо от выбора начального приближения. Это свидетельствует о хорошей устойчивости методов на данной задаче и отсутствии выраженной чувствительности к начальному положению.

Следует отметить, что в более сложных нелинейных задачах ситуация может отличаться. Метод Гаусса–Ньютона способен терять устойчивость при плохой обусловленности матрицы Якоби или при удаленности начальной точки от решения. В таких случаях регуляризация Левенберга–Марквардта обычно делает процесс оптимизации более надежным.

Вывод

Проведенное сравнение показало, что на рассматриваемой задаче методы второго порядка существенно превосходят стохастические методы по скорости сходимости и качеству найденного решения.

Метод Гаусса–Ньютона оказался самым быстрым алгоритмом и достиг минимума за две итерации. Метод Левенберга–Марквардта получил то же решение, однако потребовал больше итераций вследствие регуляризованного характера шагов.

Стохастические методы SGD и MiniBatch GD также сходятся к приемлемым решениям, однако требуют значительно большего числа итераций и обеспечивают более высокие значения функционала потерь.

Эксперимент также показал отсутствие заметной чувствительности методов Гаусса–Ньютона и Левенберга–Марквардта к выбору начального приближения на данной задаче.